PROPOSTA DE ARQUITETURA DE SOFTWARE

Aplicativo Mobile com API em Node.js e TypeScript

1. Objetivo

Este documento apresenta uma proposta inicial de arquitetura para um aplicativo mobile. A solução será organizada em camadas, separando a interface do usuário, a lógica de negócio, o banco de dados e o armazenamento de arquivos. A arquitetura poderá ser ajustada conforme as funcionalidades e os requisitos finais do projeto.

2. Tecnologias propostas

Camada	Tecnologia	Responsabilidade
Aplicativo mobile	React Native + Expo + TypeScript	Construção das telas, navegação, formulários e interação com o usuário.
API / Backend	Node.js + Express + TypeScript	Disponibilizar endpoints REST, validar requisições, aplicar regras de negócio e controlar o acesso aos dados.
Banco de dados relacional	PostgreSQL hospedado no Supabase	Armazenar dados estruturados e relacionados, como usuários, cadastros e registros do sistema.
Armazenamento de arquivos	Cloudinary	Armazenar e disponibilizar imagens, como fotos de perfil e imagens enviadas pelo aplicativo.
Versionamento	Git + GitHub	Controlar o histórico do código-fonte e facilitar o trabalho colaborativo.

3. Visão geral da arquitetura

O aplicativo mobile será responsável pela interface e enviará requisições HTTPS à API. A API, desenvolvida em Node.js com Express e TypeScript, receberá as requisições, fará as validações necessárias e executará as regras de negócio. Quando necessário, consultará ou atualizará o PostgreSQL e utilizará o serviço de armazenamento de arquivos.

APLICATIVO MOBILE **React Native + Expo + TypeScript**
↓ HTTPS / REST API · JSON ↓
API / BACKEND **Node.js + Express + TypeScript**
↓ Acesso aos dados e armazenamento

SERVIÇOS DE DADOS
PostgreSQL (Supabase) | Imagens (Cloudinary)

4. Funcionamento da comunicação

1. **1. Interação:** O usuário realiza uma ação no aplicativo, como preencher um formulário ou consultar informações.
2. **2. Requisição:** O mobile envia uma requisição à API usando HTTPS e, normalmente, dados no formato JSON.
3. **3. Validação e regras:** A API valida os dados recebidos, verifica as permissões e executa a lógica necessária.
4. **4. Persistência e arquivos:** A API consulta ou atualiza o PostgreSQL. Para imagens, o arquivo é enviado ao Cloudinary e a referência/URL é registrada no banco de dados.
5. **5. Resposta:** A API retorna uma resposta ao aplicativo, que apresenta o resultado ao usuário.

5. Organização inicial do repositório

```
projeto/
├── mobile/
│   ├── src/
│   │   ├── components/
│   │   ├── screens/
│   │   ├── navigation/
│   │   ├── services/
│   │   ├── hooks/
│   │   └── types/
│   └── app.json
├── api/
│   ├── src/
│   │   ├── config/
│   │   ├── controllers/
│   │   ├── services/
│   │   ├── repositories/
│   │   ├── models/
│   │   ├── routes/
│   │   ├── middlewares/
│   │   ├── validators/
│   │   └── server.ts
│   ├── tests/
│   └── package.json
├── docs/
│   ├── arquitetura.md
│   └── diagramas/
└── README.md
```

6. Justificativa das escolhas

- **React Native + Expo:** permite desenvolver o aplicativo mobile com TypeScript e compartilhar boa parte do código entre Android e iOS.
- **Node.js + Express + TypeScript:** oferece uma API organizada e tipada, com separação de rotas, controladores, serviços e validações.

- **PostgreSQL:** é um banco relacional adequado para dados estruturados, consultas e relacionamentos entre entidades.
- **Supabase:** fornece hospedagem gerenciada para PostgreSQL e possui uma opção gratuita sujeita a limites e condições do plano.
- **Cloudinary:** é especializado em armazenamento, entrega e processamento de imagens, com plano gratuito sujeito a cotas.
- **Git + GitHub:** permite acompanhar alterações, colaborar e manter o histórico do projeto.

7. Segurança e boas práticas

- Utilizar HTTPS nas comunicações entre o aplicativo e a API quando o sistema estiver publicado.
- Validar os dados recebidos pela API e verificar a autorização do usuário em operações protegidas.
- Manter senhas, tokens administrativos e chaves secretas no ambiente do backend, nunca embutidos no aplicativo mobile.
- Limitar tipos e tamanhos de arquivos enviados e validar imagens antes de armazená-las.
- Salvar no banco de dados os metadados e a referência do arquivo, evitando armazenar os arquivos de imagem diretamente nas tabelas do PostgreSQL.
- Configurar variáveis de ambiente e não versionar arquivos que contenham segredos.

8. Custos e limites

A proposta prioriza serviços que oferecem planos gratuitos para desenvolvimento e projetos acadêmicos. Esses planos possuem limites de armazenamento, tráfego, processamento e/ou disponibilidade, que podem mudar. Antes da publicação, será necessário conferir os termos e cotas vigentes de cada provedor e avaliar se atendem ao volume esperado do aplicativo. O uso gratuito não deve ser entendido como garantia de recursos ilimitados.

9. Links oficiais

- **React Native:** https://reactnative.dev/
- **Expo:** https://expo.dev/
- **Node.js:** https://nodejs.org/
- **Express:** https://expressjs.com/
- **TypeScript:** https://www.typescriptlang.org/
- **PostgreSQL:** https://www.postgresql.org/
- **Supabase:** https://supabase.com/
- **Cloudinary:** https://cloudinary.com/
- **GitHub:** https://github.com/

10. Observação sobre banco não relacional

Nesta proposta inicial, o PostgreSQL é o único banco de dados. O Cloudinary é um serviço de armazenamento de arquivos, não um banco de dados não relacional. Caso a disciplina exija

explicitamente a utilização de um banco NoSQL, essa necessidade deverá ser confirmada e o banco adicional deverá ser escolhido de acordo com uma funcionalidade concreta do sistema.